

分支限界法求解 0-1 背包问题

Zhesheng Xu

Student ID: 42353012

May 14th 2025

题目描述

使用分支限界法解决以下 0-1 背包问题：

背包最大承重为 16，给定 4 个商品，每个商品的重量和价值如下表：

商品编号	重量	价值
1	10	100
2	7	63
3	8	56
4	4	12

分支限界法伪代码 (Branch and Bound)

Algorithm 1 Branch and Bound 解 0-1 背包问题

Require: 背包容量 W , 物品列表 $items = [(w_1, v_1), \dots, (w_n, v_n)]$

Ensure: 最大总价值 $maxProfit$

```

1: 将物品按单位价值  $v_i/w_i$  降序排序
2: 初始化优先队列  $Q$ , 放入根结点  $u$ , 其属性为  $(level = -1, profit = 0, weight = 0)$ 
3: 设  $maxProfit \leftarrow 0$ 
4: while  $Q$  非空 do
5:   取出当前结点  $u$ 
6:   if  $u.bound > maxProfit$  then
7:     创建左子结点  $v$ , 表示 “选第  $u.level + 1$  个物品”
8:     更新  $v.weight, v.profit$  并计算其  $bound$ 
9:     if  $v.weight \leq W$  且  $v.profit > maxProfit$  then
10:       $maxProfit \leftarrow v.profit$ 
11:    end if
12:    if  $v.bound > maxProfit$  then
13:      将  $v$  加入队列
14:    end if
15:    创建右子结点  $v$ , 表示 “不选第  $u.level + 1$  个物品”
16:    设置其属性为  $u$  的当前值, 计算  $bound$ 
17:    if  $v.bound > maxProfit$  then
18:      将  $v$  加入队列
19:    end if
20:  end if
21: end while
22: return  $maxProfit$ 

```

Python 实现 (分支限界法)

```
from queue import PriorityQueue
```

```

class Node:
    def __init__(self, level, profit, weight, bound):
        self.level = level      # 当前层，即考虑到第几个物品
        self.profit = profit    # 当前总价值
        self.weight = weight    # 当前总重量
        self.bound = bound      # 当前结点的价值上界

    def __lt__(self, other):
        return self.bound > other.bound  # 优先队列中优先bound大的

def bound(u, n, W, items):
    if u.weight >= W:
        return 0
    profit_bound = u.profit
    j = u.level + 1
    total_weight = u.weight

    while j < n and total_weight + items[j][0] <= W:
        total_weight += items[j][0]
        profit_bound += items[j][1]
        j += 1

    if j < n:
        profit_bound += (W - total_weight) * items[j][1] / items[j][0]

    return profit_bound

def knapsack(items, W):
    items.sort(key=lambda x: x[1]/x[0], reverse=True)
    n = len(items)
    Q = PriorityQueue()
    u = Node(-1, 0, 0, 0)
    maxProfit = 0

```

```

u.bound = bound(u, n, W, items)
Q.put(u)

while not Q.empty():
    u = Q.get()
    if u.bound > maxProfit:
        v = Node(u.level + 1, u.profit, u.weight, 0)
        # 左孩子：选第 level+1 个物品
        v.weight += items[v.level][0]
        v.profit += items[v.level][1]
        if v.weight <= W and v.profit > maxProfit:
            maxProfit = v.profit
            v.bound = bound(v, n, W, items)
            if v.bound > maxProfit:
                Q.put(v)

        # 右孩子：不选第 level+1 个物品
        v = Node(u.level + 1, u.profit, u.weight, 0)
        v.bound = bound(v, n, W, items)
        if v.bound > maxProfit:
            Q.put(v)
    return maxProfit

items = [(10, 100), (7, 63), (8, 56), (4, 12)]
W = 16
result = knapsack(items, W)
print("最大价值为：", result)

```

算法分析

1. 基本思想

分支限界法 (Branch and Bound) 是一种系统地搜索解空间的算法，适用于解决组合优化问题。在解决背包问题时，其核心思想是通过构建一棵状

态树，并在搜索过程中使用“上界”来剪枝，避免对所有可能的组合进行穷举。

2. 时间复杂度分析

最坏情况需遍历 2^n 个节点，但实际剪枝显著减少搜索节点数。对于本题 $n = 4$ ，远小于 16 个节点被扩展。每次调用 `bound()` 的复杂度为 $O(n)$ ，总复杂度接近 $O(n \log n)$ 至 $O(2^n)$ 之间。

3. 优势与局限

优势：

- 对中小规模问题非常高效；
- 使用上界函数可有效剪枝；
- 解保证最优。

局限：

- 对大规模问题仍面临指数级节点；
- 上界估算过松时，剪枝效果下降。

4. 对本题的应用分析

对于本题，选取第 2 件和第 3 件商品（重量 $7 + 8 = 15$ ，价值 $63 + 56 = 119$ ）为最优解。分支限界法能够避开明显不合理的组合，如选择第 1 和第 2 件商品（重量为 17，超出限制）。

5. 状态空间树示意图 (TikZ)

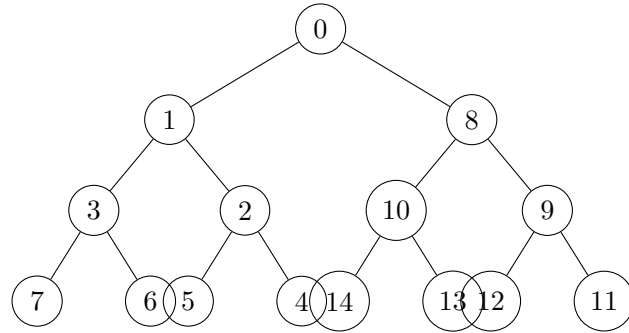


图 1: 分支限界法搜索树示意图 (节点编号示意)

该图仅为结构示意图：根节点为初始状态，每层代表是否选择当前物品的决策过程。分支限界法根据估计上界裁剪子树。